

Lesson Notes

MODULE 3 | LESSON 4

MULTI-TRANSFORMER IN ACTION

Reading Time	30 minutes
Prior Knowledge	Transformer, Multi-Transformer, LSTM, GARCH, Bagging (boosted aggregation), Ensemble learning, Feed forward network
Keywords	Multi-Head Attention

In the last lesson, we got an overview of the transformer architecture and how it can be combined with other neural networks (like LSTM) and volatility forecasting techniques. Now, we'll discuss some of the details and look at the code that makes it happen.

1. Scaled Dot-Product Attention

It was quoted in the notes for the last lesson, but it bears repeating here in proximity to the code that implements it: "The output [of the attention function] is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key" (Vaswani et al. 3). The formula in figure 1 and the visualization in figure 2 may help you see how Attention fits together with Scaled Dot Product Attention.

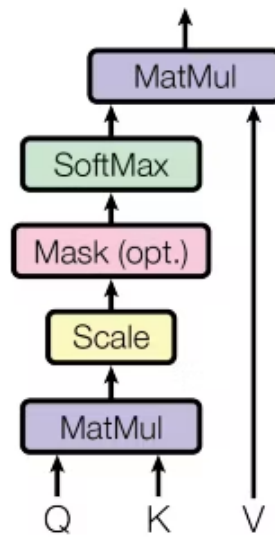
Figure 1: Attention as a Softmax Function of Dot Product

$$Attention(Q, K, V) = softmax_k \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Source: "Transformer." *TensorFlow.org*, <https://www.tensorflow.org/text/tutorials/transformer>.

Figure 2: Scaled Dot-Product Attention

Scaled Dot-Product Attention



Source: "Transformer." *TensorFlow.org*, <https://www.tensorflow.org/text/tutorials/transformer>.

Here is the relevant code:

```
def attention(self, query, key, value):

    score = tf.matmul(query, key, transpose_b=True)

    dim_key = tf.cast(tf.shape(key)[-1], tf.float32)

    scaled_score = score / tf.math.sqrt(dim_key)

    weights = tf.nn.softmax(scaled_score, axis=-1)

    output = tf.matmul(weights, value)

    return output, weights
```

(Excerpted from Ramos-Perez)

In the above, we see that the attention score is indeed a simple function of the query and key, as expected. Further, we see that the scaled score is simply divided by the square root of the dimension size (`dim_key`) or the number of features provided as inputs to the model. As described in the article by Ramos-Perez et al., this scaled score is then converted into something close to a probability using the softmax function. Finally, the output is the product of these probabilities and values.

But wait: We were told that this attention function was a simple dot product, as can be read from the title of the following illustration. Also, the output (prediction) was supposed to be a simple probability-weighted value.

Instead, the above code appears to be calculating matrix multiplication in both cases. The explanation for this apparent discrepancy lies in the fact that, given a matrix of dimension three, the dot product is calculated in batches. How many batches? That depends on the size of the first dimension of the matrices being passed to the 'attention' function. Of course, these sizes must be equal. So let us look at the definition, or more precisely, the value assignment for the variables query, key, and value, which can be found in the `call` function:

```

query = self.query_dense(inputs) # (batch_size, seq_len, embed_dim)

key = self.key_dense(inputs) # (batch_size, seq_len, embed_dim)

value = self.value_dense(inputs) # (batch_size, seq_len, embed_dim)

```

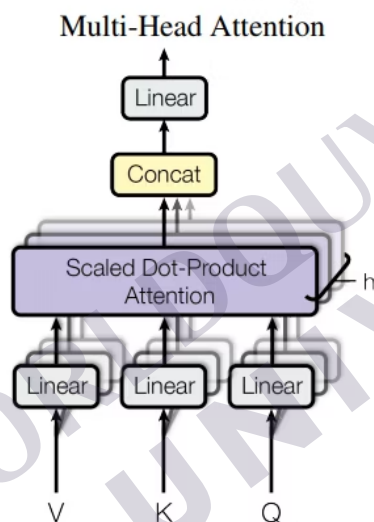
(Excerpted from Ramos-Perez)

Alas, we see from the comments that each of these matrices is indeed three-dimensional and that the first dimension of each is the same (batch_size). According to this understanding, we do not end up with a single result but rather a batch_size number of results.

2. Multi-Head Attention

Our discussion in the last lesson notes only discussed (implicitly) the calculation of a single attention, but as you read in the article describing this model and as illustrated below for the “Linear” and “Scaled Dot-Product Attention” layers, we are actually dealing with multi-head attention. In the diagram, we see h=3 attention calculations, which are simply concatenated.

Figure 3: Concatenation of Attentions: Multi-Head Attention



Source: "Transformer." *TensorFlow.org*, <https://www.tensorflow.org/text/tutorials/transformer>.

This diagram perhaps better illustrates the equation from Vaswani et al., though it may help to see them together:

Figure 4: Formula for Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{where } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

Source: Koch, Brando. "Attention is All You Need Paper." *GitHub*, <https://github.com/bkoch4142/attention-is-all-you-need-paper>.

Finally, we now see the code, which implements this idea of combining attentions through concatenation (this can be found later in the same function as cited above):

```

concat_attention = tf.reshape(attention, (batch_size, -1, self.embed_dim)) # (batch_

output = self.combine_heads(concat_attention) # (batch_size, seq_len, embed_dim)

```

(Excerpted from Ramos-Perez)

Why do we calculate the same thing h number of times? In the notes for the last lesson, we discussed the projection of inputs into a feature space resulting in a feature vector, which ideally captured the most important information for the prediction. In a similar way, each query is linearly projected into a feature space, but as the optimization of this feature space (the training of the relevant parameter weights) exhibits some randomness, we improve our chances of meaningful feature extraction by performing the procedure multiple times.

3. Multi-Transformer: The Transformer Extended

You might guess that Ramos-Perez et al. saw the wisdom of multiple attempts at prediction and applied it at an even larger scale: If multi-head attention is better than simple (single-head) attention, then why would a multi-transformer model not outperform a simple, single transformer model? In retrospect, it seems trivial, but nevertheless, such a simple extension of the model in this way indeed provides for more accurate results according to their findings (Ramos-Perez et al. 12–14).

Assuming the most likely case, that is, that we need a single VaR prediction, how can we optimally make use of the multiple transformers in play here? Ramos-Perez et al. employ the bagging (bootstrap aggregating) approach, whereby each of the five transformers takes a random sample of 90% of the available observations as its input. The average of the outputs of these five transformers should therefore benefit from the usual advantages of bagging: It should be less prone to overfitting and result in less variance.

4. Conclusion

In this module, we used very sophisticated neural network architectures to better estimate risk values such as VaR. In the next module, we explore how extreme value theory can better improve our understanding of distribution tails and better predict extreme losses.

References

- Koch, Brando. "Attention is All You Need Paper." *GitHub*. <https://github.com/bkoch4142/attention-is-all-you-need-paper>.
- Ramos-Perez, Eduardo, et al. "Multi-Transformer: A New Neural Network-Based Architecture for Forecasting S&P Volatility." *Mathematics*, vol. 9, no. 15. <https://www.mdpi.com/2227-7390/9/15/1794>.
- Ramos-Perez, Eduardo. "Multi-Transformer: A New Neural Network-Based Architecture for Forecasting S&P Volatility." *GitHub*. <https://github.com/EduardoRamosP/MultiTransformer>.
- "Transformer." *TensorFlow.org*. <https://www.tensorflow.org/text/tutorials/transformer>.